

**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-4

CH.SC.U4CSE24221

TEJESH KANTA

1.PRIMS

CODE:

```
C prims.c > main()
1  #include <stdio.h>
2  #define INF 999
3
4  int main() {
5      int n, cost[10][10];
6      int visited[10] = {0};
7      int i, j, ne = 1;
8      int min, a = 0, b = 0;
9      int mincost = 0;
10
11     printf("TEJESH KANTA-CH.SC.U4CSE24221");
12
13     printf("Enter number of vertices: ");
14     scanf("%d", &n);
15
16     printf("Enter cost adjacency matrix:\n");
17     for (i = 1; i <= n; i++) {
18         for (j = 1; j <= n; j++) {
19             scanf("%d", &cost[i][j]);
20             if (cost[i][j] == 0)
21                 cost[i][j] = INF;
22         }
23     }
24
25     visited[1] = 1;
26
27     printf("\nEdges in Minimum Spanning Tree:\n");
28
29     while (ne < n) {
30         min = INF;
31         for (i = 1; i <= n; i++) {
32             if (visited[i]) {
33                 for (j = 1; j <= n; j++) {
34                     if (!visited[j] && cost[i][j] < min) {
35                         min = cost[i][j];
36                         a = i;
37                         b = j;
```

```

37         b = j;
38     }
39 }
40 }
41 }
42
43     printf("%d -> %d = %d\n", a, b, min);
44     mincost += min;
45     visited[b] = 1;
46     ne++;
47 }
48
49 printf("\nMinimum cost = %d\n", mincost);
50 return 0;
51 }

```

OUTPUT:

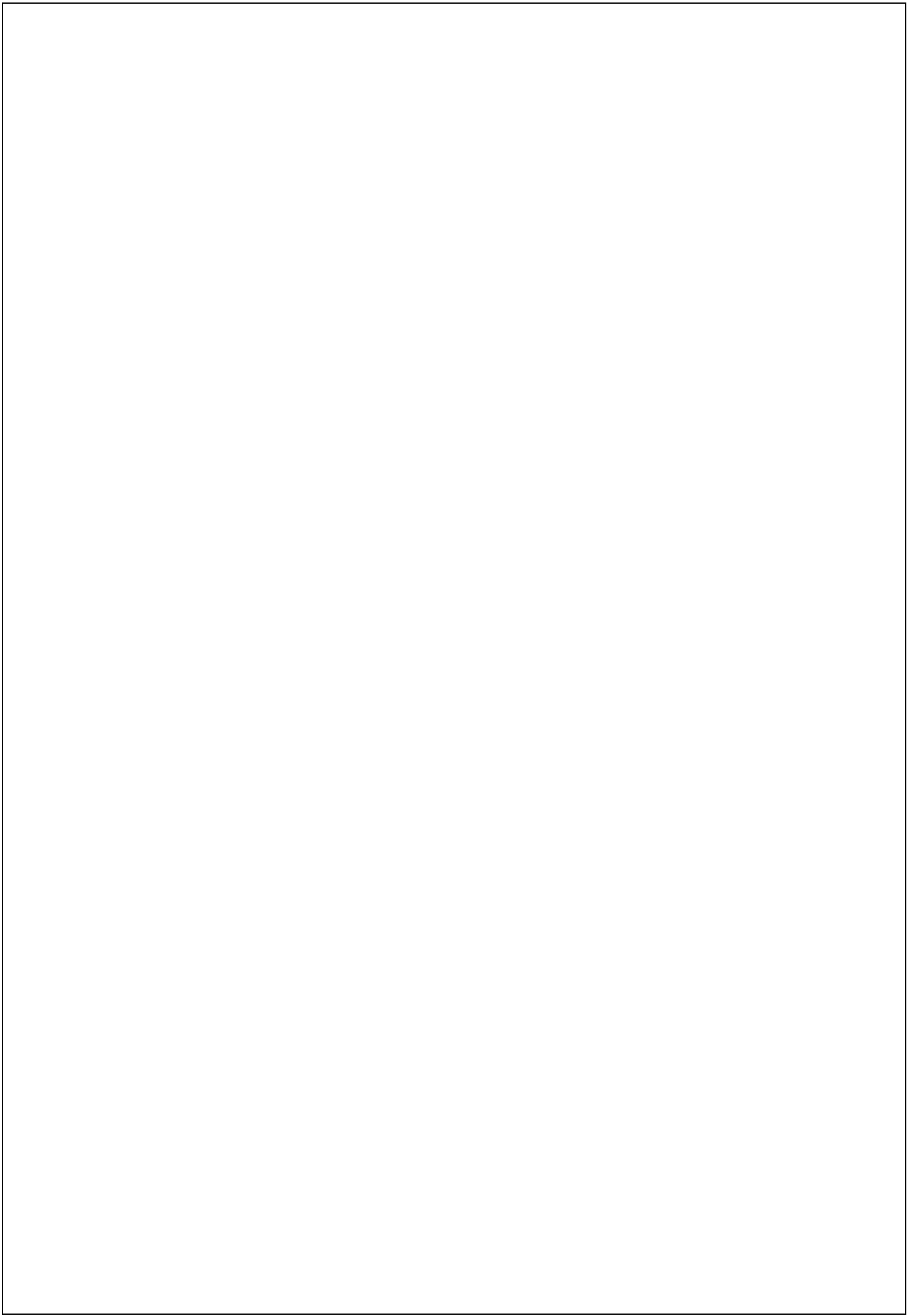
```

[kantatejesh@KANTAs-MacBook-Air Desktop % cd Website
[kantatejesh@KANTAs-MacBook-Air Website % gcc prims.c -o prims
[kantatejesh@KANTAs-MacBook-Air Website % ./prims
TEJESH KANTA-CH.SC.U4CSE24221Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0

Edges in Minimum Spanning Tree:
1 -> 4 = 5
4 -> 3 = 4
1 -> 2 = 10

Minimum cost = 19

```



2.KRUSKAL

CODE:

```
C kruskal.c > main()
1  #include <stdio.h>
2
3  int parent[10];
4
5  int find(int i) {
6      while (parent[i])
7          i = parent[i];
8      return i;
9  }
10
11 int uni(int i, int j) {
12     if (i != j) {
13         parent[j] = i;
14         return 1;
15     }
16     return 0;
17 }
18
19 int main() {
20     int n, cost[10][10];
21     int i, j, ne = 1;
22     int min, a = 0, b = 0;
23     int mincost = 0;
24     printf("TEJESH KANTA-CH.SC.U4CSE24221");
25
26     printf("Enter number of vertices: ");
27     scanf("%d", &n);
28
29     printf("Enter cost adjacency matrix:\n");
30     for (i = 1; i <= n; i++) {
31         for (j = 1; j <= n; j++) {
32             scanf("%d", &cost[i][j]);
33             if (cost[i][j] == 0)
34                 cost[i][j] = 999;
35         }
36     }
37 }
```

```

37
38     printf("\nEdges in Minimum Spanning Tree
39
40     while (ne < n) {
41         min = 999;
42         for (i = 1; i <= n; i++) {
43             for (j = 1; j <= n; j++) {
44                 if (cost[i][j] < min) {
45                     min = cost[i][j];
46                     a = i;
47                     b = j;
48                 }
49             }
50         }
51
52         int u = find(a);
53         int v = find(b);
54
55         if (uni(u, v)) {
56             printf("%d -> %d = %d\n", a, b,
57                 mincost += min;
58                 ne++;
59         }
60
61         cost[a][b] = cost[b][a] = 999;

```

OUTPUT:

```

[kantatejesh@KANTAs-MacBook-Air Website % ./kruska
TEJESH KANTA-CH.SC.U4CSE24221Enter number of vert
Enter cost adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in Minimum Spanning Tree:
1 -> 2 = 2
2 -> 3 = 3
2 -> 5 = 5
1 -> 4 = 6

```

3.AVL

CODE:

```
Users > kantatejesh > Desktop > DAA LAB > TASK4 > C AVL.c > ...
1  //CH.SC.U4CSE24221
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  struct Node {
6      int key;
7      struct Node *left, *right;
8      int height;
9  };
10
11  int max(int a, int b) {
12      return (a > b) ? a : b;
13  }
14
15  int getHeight(struct Node *n) {
16      return (n == NULL) ? 0 : n->height;
17  }
18
19  struct Node* newNode(int key) {
20      struct Node* node = (struct Node*)malloc(sizeof(struct Node));
21      node->key = key;
22      node->left = node->right = NULL;
23      node->height = 1;
24      return node;
25  }
26
27  struct Node* rightRotate(struct Node *y) {
28      struct Node *x = y->left;
29      struct Node *T2 = x->right;
30
31      x->right = y;
32      y->left = T2;
33
34      y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
35      x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
36
37      return x;
38  }
39
40  struct Node* leftRotate(struct Node *x) {
```

```

41     struct Node *y = x->right;
42     struct Node *T2 = y->left;
43
44     y->left = x;
45     x->right = T2;
46
47     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
48     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
49
50     return y;
51 }
52
53 struct Node* balanceTree(struct Node* node) {
54     if (node == NULL)
55         return NULL;
56
57     node->left = balanceTree(node->left);
58     node->right = balanceTree(node->right);
59
60     node->height = 1 + max(getHeight(node->left), getHeight(node->right));
61
62     int balance = getHeight(node->left) - getHeight(node->right);
63
64     if (balance > 1 && getHeight(node->left->left) >= getHeight(node->left->right))
65         return rightRotate(node);
66
67     if (balance > 1 && getHeight(node->left->left) < getHeight(node->left->right)) {
68         node->left = leftRotate(node->left);
69         return rightRotate(node);
70     }
71
72     if (balance < -1 && getHeight(node->right->right) >= getHeight(node->right->left))
73         return leftRotate(node);
74
75     if (balance < -1 && getHeight(node->right->right) < getHeight(node->right->left)) {
76         node->right = rightRotate(node->right);
77         return leftRotate(node);
78     }
79 }

```



```

80     return node;
81 }
82
83 void printGivenLevel(struct Node* root, int level) {
84     if (root == NULL)
85         return;
86
87     if (level == 1)
88         printf("%d ", root->key);
89     else {
90         printGivenLevel(root->left, level - 1);
91         printGivenLevel(root->right, level - 1);
92     }
93 }
94
95 void printLevelOrder(struct Node* root) {
96     int h = getHeight(root);
97     for (int i = 1; i <= h; i++)
98         printGivenLevel(root, i);
99 }
100
101 int main() {
102
103     struct Node *root = newNode(157);
104     root->left = newNode(110);
105     root->left->right = newNode(147);
106     root->left->right->left = newNode(122);
107     root->left->right->right = newNode(149);
108     root->left->right->right->right = newNode(151);
109     root->left->right->left->left = newNode(111);
110     root->left->right->left->right = newNode(141);
111     root->left->right->left->left->right = newNode(112);
112     root->left->right->left->right->left = newNode(123);
113     root->left->right->left->right->left->right = newNode(133);
114     root->left->right->left->right->left->left = newNode(117);
115
116     root = balanceTree(root);
117
118     printf("AVL Level Order: ");

```

OUTPUT:

AVL Level Order: 123 110 157 122 147 111 117 141 149 112
133 151

CH.SC.U4CSE24221

Space Complexity and it's Justification:

Time Complexity

$O(n)$

Justification: Each node is visited once; height check and rotations take $O(1)$ time.

Space Complexity

$O(h) \rightarrow O(\log n)$ (balanced), $O(n)$ (skewed)

Justification: Space used only by recursion stack, proportional to tree height.

4. RED

BLACK

TREE

CODE:

```
Users > kantatejesh > Desktop > DAA LAB > TASK4 > C RB.c > ...
1  //CH.SC.U4CSE24221
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  enum Color { RED, BLACK };
6
7  struct Node {
8      int data;
9      enum Color color;
10     struct Node *left, *right, *parent;
11 };
12
13 struct Node* newNode(int data) {
14     struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
15     temp->data = data;
16     temp->left = temp->right = temp->parent = NULL;
17     temp->color = RED;
18     return temp;
19 }
20
21 void rotateLeft(struct Node** root, struct Node* x) {
22     struct Node* y = x->right;
23     x->right = y->left;
24     if (y->left != NULL)
25         y->left->parent = x;
26
27     y->parent = x->parent;
28
29     if (x->parent == NULL)
30         *root = y;
31     else if (x == x->parent->left)
32         x->parent->left = y;
33     else
34         x->parent->right = y;
35
36     y->left = x;
37     x->parent = y;
38 }
39
40 void rotateRight(struct Node** root, struct Node* y) {
```

```

40 void rotateRight(struct Node** root, struct Node* y) {
41     struct Node* x = y->left;
42     y->left = x->right;
43     if (x->right != NULL)
44         x->right->parent = y;
45
46     x->parent = y->parent;
47
48     if (y->parent == NULL)
49         *root = x;
50     else if (y == y->parent->left)
51         y->parent->left = x;
52     else
53         y->parent->right = x;
54
55     x->right = y;
56     y->parent = x;
57 }
58
59 void fixViolation(struct Node** root, struct Node* z) {
60     while (z != *root && z->parent->color == RED) {
61
62         if (z->parent == z->parent->parent->left) {
63             struct Node* y = z->parent->parent->right;
64
65             if (y != NULL && y->color == RED) {
66                 z->parent->color = BLACK;
67                 y->color = BLACK;
68                 z->parent->parent->color = RED;
69                 z = z->parent->parent;
70             } else {
71                 if (z == z->parent->right) {
72                     z = z->parent;
73                     rotateLeft(root, z);
74                 }
75                 z->parent->color = BLACK;
76                 z->parent->parent->color = RED;
77                 rotateRight(root, z->parent->parent);
78             }
79         } else {

```

```

79         } else {
80             struct Node* y = z->parent->parent->left;
81
82             if (y != NULL && y->color == RED) {
83                 z->parent->color = BLACK;
84                 y->color = BLACK;
85                 z->parent->parent->color = RED;
86                 z = z->parent->parent;
87             } else {
88                 if (z == z->parent->left) {
89                     z = z->parent;
90                     rotateRight(root, z);
91                 }
92                 z->parent->color = BLACK;
93                 z->parent->parent->color = RED;
94                 rotateLeft(root, z->parent->parent);
95             }
96         }
97     }
98     (*root)->color = BLACK;
99 }
100
101 int height(struct Node* node) {
102     if (node == NULL)
103         return 0;
104     int l = height(node->left);
105     int r = height(node->right);
106     return (l > r ? l : r) + 1;
107 }
108
109 void printGivenLevel(struct Node* root, int level) {
110     if (root == NULL)
111         return;
112
113     if (level == 1)
114         printf("%d ", root->data);
115     else {
116         printGivenLevel(root->left, level - 1);

```

```
128
129 int main() {
130
131     struct Node* root = newNode(157);
132     root->color = BLACK;
133
134     struct Node* n110 = newNode(110);
135     root->left = n110; n110->parent = root;
136
137     struct Node* n147 = newNode(147);
138     n110->right = n147; n147->parent = n110;
139
140     struct Node* n122 = newNode(122);
141     n147->left = n122; n122->parent = n147;
142
143     struct Node* n149 = newNode(149);
144     n147->right = n149; n149->parent = n147;
145
146     struct Node* n111 = newNode(111);
147     n122->left = n111; n111->parent = n122;
148
149     struct Node* n141 = newNode(141);
150     n122->right = n141; n141->parent = n122;
151
152     struct Node* n151 = newNode(151);
153     n149->right = n151; n151->parent = n149;
154
155     struct Node* n112 = newNode(112);
156     n111->right = n112; n112->parent = n111;
157
158     struct Node* n123 = newNode(123);
159     n141->left = n123; n123->parent = n141;
160
161     struct Node* n117 = newNode(117);
162     n112->left = n117; n117->parent = n112;
163
164     struct Node* n133 = newNode(133);
165     n123->right = n133; n133->parent = n123;
166
```

```

166
167     fixViolation(&root, n147);
168     fixViolation(&root, n111);
169     fixViolation(&root, n141);
170     fixViolation(&root, n151);
171     fixViolation(&root, n117);
172     fixViolation(&root, n133);
173
174     printf("Red-Black Level Order: ");
175     printLevelOrder(root);
176     printf("\n");
177     printf("CH.SC.U4CSE24221\n");
178     return 0;
179 }
180
181

```

OUTPUT:

```

Red-Black Level Order: 122 | 111 147 | 110 112 133 151 |
    117 123 141 149 157 |
CH.SC.U4CSE24221

```

Space Complexity and Its Justification:

Time Complexity

$O(\log n)$

Justification:

Red-Black Tree height is always $O(\log n)$.

Insertion fix (fixViolation) does at most constant rotations and recolorings while moving up the tree.

Space Complexity

$O(h) \rightarrow O(\log n)$

Justification:

Uses recursion only for traversal/height, and recursion depth equals tree height.

No extra data structures are used.